

Article

DynER: Optimized Test Case Generation for Representational State Transfer (REST)ful Application Programming Interface (API) Fuzzers Guided by Dynamic Error Responses

Juxing Chen , Yuanchao Chen , Zulie Pan * , Yu Chen, Yuwei Li, Yang Li, Min Zhang * and Yi Shen

College of Electronic Engineering, National University of Defense Technology, Hefei 230037, China; chenjuxing@nudt.edu.cn (J.C.); chenyuanchao@nudt.edu.cn (Y.C.); cy@nudt.edu.cn (Y.C.); liyuwei@nudt.edu.cn (Y.L.); liyanghf@nudt.edu.cn (Y.L.); shenyl@nudt.edu.cn (Y.S.)

* Correspondence: panzulie17@nudt.edu.cn (Z.P.); zhangmindy@nudt.edu.cn (M.Z.)

Abstract: Modern web services widely provide RESTful APIs for clients to access their functionality programmatically. Fuzzing is an emerging technique for ensuring the reliability of RESTful APIs. However, the existing RESTful API fuzzers repeatedly generate invalid requests due to unawareness of errors in the invalid tested requests and lack of effective strategy to generate legal value for the incorrect parameters. Such limitations severely hinder the fuzzing performance. In this paper, we propose DynER, a new test case generation method guided by dynamic error responses during fuzzing. DynER designs two strategies of parameter value generation for purposefully revising the incorrect parameters of invalid tested requests to generate new test requests. The strategies are, respectively, based on prompting Large Language Model (LLM) to understand the semantics information in error responses and actively accessing API-related resources. We apply DynER to the state-of-the-art fuzzer RESTler and implement DynER-RESTler. DynER-RESTler outperforms foREST on two real-world RESTful services, WordPress and GitLab with a 41.21% and 26.33% higher average pass rate for test requests and a 12.50% and 22.80% higher average number of unique request types successfully tested, respectively. The experimental results demonstrate that DynER significantly improves the effectiveness of test cases and fuzzing performance. Additionally, DynER-RESTler finds three new bugs.

Keywords: RESTful API; test case generation; LLM for fuzzing; automated software testing; web service



Citation: Chen, J.; Chen, Y.; Pan, Z.; Chen, Y.; Li, Y.; Li, Y.; Zhang, M.; Shen, Y. DynER: Optimized Test Case Generation for Representational State Transfer (REST)ful Application Programming Interface (API) Fuzzers Guided by Dynamic Error Responses. *Electronics* **2024**, *13*, 3476. <https://doi.org/10.3390/electronics13173476>

Academic Editor: Porfirio Tramontana

Received: 1 August 2024

Revised: 28 August 2024

Accepted: 30 August 2024

Published: 1 September 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

RESTful APIs are becoming the most popular endpoints for programmatically accessing web services or cloud services [1]. Most web service providers, such as Google [2] and Amazon [3], use RESTful APIs to grant access to third-party applications or services. As RESTful APIs gain popularity, their functionality, reliability and security have attracted widespread attention in recent years [4].

Automatically testing web services via their RESTful APIs and checking whether those services are reliable and secure is thriving. RESTful API fuzzing is one of the most prevalent approaches, and many new solutions have been proposed in the past years [5–13]. Given standard API specifications, usually with the OpenAPI [14] specifications (OAS) as input, a RESTful API fuzzer generates test cases automatically and exercises each test case with the service under test (SUT). Many existing studies [5–8] focus on how to construct the correct test request sequences by inferring the dependencies between different request types. Others [10–13] focus on how to generate each test request by assigning a value to the parameters of each request type. However, existing fuzzers still generate many invalid requests that are rejected by the SUTs and fail to exercise service functionality in depth, leading to poor fuzzing performance.

We summarize two main limitations in the existing test case generation that lead to state-of-the-art RESTful API fuzzers failing to generate high-quality test requests. (1) The existing test case generators are unaware of errors in the invalid tested requests, resulting in subsequently generating more new invalid test requests that contain the same incorrect parameters. (2) With the existing strategies of parameter value generation, it is hard to generate legal values for these incorrect parameters due to the lack of effective information related to them. Only relying on the information extracted from the OAS is not enough to render legal values for them.

In this paper, we propose a novel optimized test case generation method guided by dynamic error responses, named DynER, aiming to overcome the above two limitations and improve the effectiveness of test cases. Based on the invalid tested requests and their error response requests received from SUTs, DynER purposefully revises the errors of invalid tested requests to generate new test requests that are more likely to be valid ones.

DynER captures invalid requests and analyzes the reason of errors based on the error response requests received from SUT during the fuzzing process. To revise the incorrect parameter values in invalid test requests, DynER designs two novel strategies to generate legal value for these parameters. The first strategy is motivated by the semantic information conveyed by error responses that identifies the incorrect parameters and describes their legal value formats and constraints. Additionally inspired by the semantic understanding capabilities of pre-trained large language models (LLMs), this strategy prompts LLMs to revise the incorrect part of invalid tested requests according to semantic information through prompt engineering. Motivated by the resource design features of RESTful API, the second strategy actively accesses the API-related resource information on SUTs to obtain the candidate values for the resource ID parameters or those with little or no description.

Notably, DynER is a generic scheme that can be applied to various RESTful API fuzzers to optimize their test case generation and improve their fuzzing performance. We apply DynER to a state-of-the-art fuzzer RESTler [5] and implement DynER-RESTler. We conduct a set of experiments to evaluate the performance of DynER-RESTler against RESTler and foREST [8] on two open-source services, GitLab [15] and WordPress [16], via 15 representative RESTful APIs. The experimental results firstly demonstrate that DynER can significantly improve test case generation for RESTler to generate more valid test cases. Furthermore, we evaluate DynER-RESTler against RESTler and foREST for overall fuzzing performance on the real-world RESTful services. The average pass rate of test requests generated by DynER-RESTler is 41.21% and 26.33% higher than foREST on WordPress and GitLab. The average number of unique request types successfully tested by DynER-RESTler is 12.5% higher on WordPress and 22.80% higher on GitLab compared to foREST. The results indicate that DynER-RESTler significantly improves the test request generation to pass the service's checking and tests the SUTs more comprehensively and deeply. Additionally, DynER-RESTler found three new bugs that were previously unknown.

The main contributions of this paper are summarized as follows.

- We propose a novel test case generation method named DynER. DynER generates new test requests by purposefully revising the errors of invalid tested requests guided by dynamic error responses. DynER designs two new strategies for generating legal parameter values, which explore the potential of LLMs in assisting fuzzers in understanding dynamic error responses and exploit the resource design features of RESTful API to actively access resource information from the SUTs.
- We apply DynER to the state-of-the-art fuzzer RESTler and implement DynER-RESTler. DynER can be generally applied to a broad range of existing RESTful API fuzzers and facilitate future research on RESTful API fuzzing.
- We evaluate DynER-RESTler against RESTler and foREST on real-world RESTful services. The results demonstrate the significant performance of DynER on high-quality test request generation. DynER can significantly optimize the test case generation and improve fuzzing performance for RESTful API fuzzers.

The paper is organized as follows. The related work is discussed in Section 2. Section 3 provides background information to better understand the rest of paper. In Section 4, we introduce our motivation to better illustrate the problem we are addressing in this article. The proposed approaches are presented in Section 5. The empirical study and experiment results are shown in Section 6. We discuss threats to validity and future work in Section 7 and conclude the article in Section 8.

2. Related Works

The widespread use and distinctive characteristics of RESTful APIs have recently drawn significant attention from the testing research community. In this section, we provide an overview of the related works in automated test generation for RESTful APIs, with a particular focus on the existing methods for parameter value generation. Additionally, we introduce related research on using LLMs for fuzzing.

2.1. Automated Test Generation for RESTful API

For RESTful APIs, most of the existing works related to automated test generation employ **black-box testing**.

In 2019, Atlidakis et al. proposed RESTler [5], the first stateful black-box RESTful API fuzzer designed to automatically test cloud services via their RESTful APIs. RESTler is a generation-based fuzzer that uses a predefined dictionary to instantiate parameter values and analyzes input OpenAPI specifications statically to infer dependencies between request types. Furthermore, Atlidakis et al. implemented several security rule checkers [17] in RESTler that can automatically detect violations of these security rules. To further enhance RESTler's capabilities, Godefroid et al. defined differential regression testing for RESTful APIs, which leverages RESTler to construct network logs on different versions of the target RESTful APIs and detects service and specification regressions by comparing these network logs [18]. Godefroid et al. also studied how to generate data payloads learned from RESTful API specifications and find data-processing bugs in cloud services [19].

Model-based testing is a representative approach for automatically generating test cases for RESTful APIs. Viglianisi et al. proposed RESTTESTGEN [6], which considers data dependencies among operations and operation semantics. The dependencies between APIs are specified using an operation dependency graph (ODG), which is initialized with an OpenAPI schema and evolves during test execution. Liu et al. propose Morest [7], which utilizes a dynamically updating RESTful-service property graph (RPG) to model the dependencies. Compared to the ODG, the RPG not only models the producer–consumer dependencies between APIs with more detail but also captures property equivalence relations between schemas. This allows the RPG to describe a broader range of RESTful service behaviors and flexibly update itself based on execution feedback.

Karlsson et al. proposed QuickREST [20], a property-based test generation approach using OpenAPI schemas. QuickREST checks for non-500 status codes and schema compliance. Hatfield-Dodds et al. presented Schemathesis [21], a tool for finding semantic errors and crashes in OpenAPI or GraphQL web APIs through property-based testing.

Tsai et al. proposed HsuanFuzz [22], which introduced the new concept of “RESTful API black-box fuzz testing based on coverage level guidelines”. HsuanFuzz uses test coverage level [23] as feedback to guide black-box fuzzers. Additionally, HsuanFuzz employs pairwise testing to reduce the combinations of test parameters and accelerate the testing process. Wu et al. present RestCT [10], a systematic and fully automated approach that adopts combinatorial testing (CT) to test RESTful APIs. RestCT is systematic in that it not only covers and tests the interactions of a certain number of operations in RESTful APIs but also the interactions of specific input parameters within each operation.

Very recently, Lin et al. proposed foREST [8], a novel tree-based RESTful API black-box fuzzing approach that was more efficient than the traditional graph-based approaches. foREST subtly models the relations of APIs with a tree structure that not only reduces the complexity of API dependencies but also captures the priority of resource dependencies.

Fuzzing experiments with real-world RESTful services show that foREST achieves better performance than state-of-the-art tools.

There are a few studies that employ **white-box** information (e.g., code coverage) to assist automated test generation for RESTful APIs.

Atlidakis et al. proposed Pythia [24], a grey-box fuzzer that uses the built-in functions of Ruby to obtain code coverage. Pythia is designed based on the execution results of RESTler. The mutation stage is improved using a machine learning method, while the validity of the grammar is maintained even after some noise is injected. Arcuri et al. considered testing from a developer's perspective and designed the white-box testing tool EvoMASTER [25]. EvoMASTER uses genetic algorithms to generate random test cases and guide mutation testing based on coverage. Subsequently, Arcuri et al. [26] improved EvoMASTER by employing the MIO algorithm for intelligent sampling to generate pre-defined structured tests, thus accelerating the search process. However, the enhanced EvoMASTER still exhibits low code coverage in certain projects, leaving room for further improvement. To address this, they [27] take the database state into account by monitoring SQL commands and calculating heuristic values to enhance search-based testing. They continuously optimize and improve EvoMASTER with an adaptive weight-based hypermutation approach [28] and extend Rd-MIO* by employing SQL commands [29].

Regarding **concrete parameter value generation**, using static mapping directories and random values are likely the simplest and most common approaches in the existing research [5,19]. RestTestGen [6] proposes four sources for generating valid parameter values, including values from a previous successful request, parameter dictionaries, examples from the documentation, or completely random values. The subsequent work mostly utilizes the aforementioned conventional methods to generate request parameter values. According to the evaluation results of 10 RESTful API fuzzers conducted by Kim et al. [30], these tools generate many invalid requests that are rejected by the services and fail to exercise service functionality in depth. In most cases, this occurs when the parameters require domain-specific values or have data-format restrictions.

Several recent works focus on generating valid parameter values to improve the effectiveness of test requests. Alberto et al. proposed an automated testing tool, RESTest [11], based on their previous work where they introduced an IDL for formalizing request parameter constraint relationships and an automated analysis tool [31]. RESTest enables the automatic analysis of internal parameter constraint relationships and generates parameter values that satisfy the constraints. RestCT [10] proposes a pattern-matching approach to infer parameter constraint relationships and then uses combinatorial testing methods to construct request parameter constraint coverage arrays, thereby instantiating request parameter values. However, since RestCT relies on a set of hard-coded pattern matching, it lacks flexibility and generality. NLPtoREST [12] proposes using natural language processing techniques to automatically extract parameter constraint relationships described in natural language in OpenAPI documentation and add these constraints to OpenAPI documents following OpenAPI syntax to assist in generating valid request parameter values during fuzz testing. Alonso et al. [13] considered the practical significance of parameters and used natural language processing technology to extract data from a knowledge base, improving the automation of parameter value generation. Instead of generating valid parameter values, Mirabella et al. [32] proposed a deep-learning-based approach to automatically infer whether a generated request is valid, i.e., whether the request satisfies all parameter constraints, before dynamic testing.

Existing test case generation still has two main **limitations**, which cause state-of-the-art RESTful API fuzzers to generate many invalid test requests during fuzzing. On the one hand, existing test case generators are unaware of the errors in invalid test requests, leading to the subsequent generation of more invalid test requests containing the same incorrect parameters. Instead of reassigning all parameters of the request template each time, we propose an innovative approach to generate new test requests by revising the incorrect parts of previously tested invalid requests. On the other hand, existing strategies for parameter

value generation struggle to produce valid values for incorrect parameters due to a lack of effective information. Relying solely on information extracted from the OAS is insufficient to generate valid values for these incorrect parameters. These approaches overlook useful parameter description information in the error responses of invalid test requests. Moreover, they do not attempt to utilize the existing API-related resource information in the target services. In this paper, we design two new strategies for generating valid parameter values, based on understanding the semantic information of dynamic error responses and actively accessing API-related resource information from the SUTs.

2.2. Large Language Model for Fuzzing

Emerging LLMs have demonstrated impressive performance on many very specific downstream tasks using prompt engineering [33], where users provide the LLMs with a description of the task and, optionally, a few examples of solving the task. More recently, researchers have explored the capacity of large language models (LLMs) in the field of fuzzing.

Lemieux et al. proposed CODAMOSA [34], the first to apply LLMs to fuzzing. CODAMOSA demonstrates the capability of LLMs in the automatic generation of test cases for Python modules. For fuzzing deep learning libraries such as PyTorch and TensorFlow, Deng et al. proposed TitanFuzz [35], which uses Codex [36] to generate seed programs and InCoder [37] to perform template-based mutation. Deng et al. also proposed FuzzGPT [38], which utilizes historical bug-triggering code snippets to either prompt or fine-tune LLMs for generating more unusual code snippets, leading to more effective fuzzing. Ackerman et al. [39] leveraged LLMs for fuzzing parsers by employing LLMs to recursively examine a natural language format specification and generate instances from the specification to use as strong seed examples for a mutation fuzzer. Meng et al. proposed CHATAFL [40], an LLM-guided fuzzing engine for protocol implementations, to overcome the challenges of the existing protocol fuzzers. CHATAFL uses LLMs' knowledge of protocol message types for well-known protocols. Cheng et al. proposed MSFuzz [41], which leverages LLMs to augment protocol fuzzing with message syntax comprehension. Wang et al. proposed LLMIF [42], a fuzzing algorithm that incorporates LLMs into IoT fuzzing. Xia et al. proposed Fuzz4All [43], which leverages LLMs as an input generation and mutation engine. Fuzz4All enables the generation of diverse and realistic inputs for any practically relevant language.

Two contemporaneous studies explore the capability of LLMs in RESTful API testing. Decrop et al. proposed RESTSpecIT [44], the first automated RESTful API specification inference and black-box testing approach leveraging LLMs. Kim et al. proposed RESTGPT [45], which harnesses large language models (LLMs) to enhance REST API specifications by identifying constraints and generating relevant parameter values. Unlike their work, we leverage LLMs to understand the semantic information in error responses during dynamic testing and revise invalid parameters. In other words, we apply LLMs to specialized tasks from a different perspective to enhance RESTful API fuzzing.

3. Background

3.1. RESTful API

REST is an acronym for representational state transfer and an architectural style for distributed hypermedia systems [46]. Roy Fielding first presented it in 2000 in his famous dissertation [47]. Since then, it has become one of the most widely used approaches for building web-based application programming interfaces (APIs) [48].

In REST, the primary data representation is called resource. A resource can be a singleton or a collection. For example, "customers" is a collection resource and "customer" is a singleton resource (in a banking domain). RESTful APIs use uniform resource identifiers (URIs) to address resources. We can identify "customers" collection resources using the URI "/customers". We can identify a single "customer" resource using the URI "/customers/customerId".

REST is almost synonymous with hypertext transfer protocol (HTTP), which is the foundational protocol of the World Wide Web. RESTful APIs enable users to develop all kinds of web applications with all possible create, retrieve, update, delete (CRUD) operations. REST guidelines recommend applying a specific HTTP method for each type of request made to the server. While it is technically possible to deviate from this guideline, it is strongly discouraged. For example, use GET requests to retrieve resource information only and use POST requests to create resources.

The OpenAPI defines a formal standard for describing RESTful API. The documentation that follows this standard is called OpenAPI specification (OAS). The OpenAPI specification of the target RESTful service contains the information concerning the object schemas as well as the API endpoints of a service, including but not limited to the available CRUD operations and input parameters, as well as expected responses. Each object has pre-defined fields and corresponding parameter types. Users can follow the specification to produce valid API operations and render them into HTTP requests to interact with the RESTful service endpoints. According to OAS, both people or computers could launch HTTP requests to call the RESTful APIs and make use of the functionality that APIs provide.

3.2. RESTful API Fuzzing

The RESTful API fuzzer automatically generates test cases based on the API specification of the RESTful service. Then, it sends these test cases to the target service to explore potential flaws. If a test case triggers an HTTP response code in the 50X range of the target service [49], the fuzzer will assume that an error has occurred and store the test case for further analysis. The main process of RESTful API fuzzing is as follows.

Processing API specifications. Based on an OpenAPI specification as user input, the task of this stage is to automatically construct request templates for each API request type and infer the dependencies among different API request types. The request templates contain the information of each parameter, which will be used to generate concrete HTTP requests. It is worth noting that the input OAS is always written manually according to the usage description for RESTful APIs on the official website of each target service. Thus, its correctness and completeness cannot be ensured.

Test case generation. The stage of test case generation usually contains two tasks: constructing request sequences and then instantiating every abstract request template in each sequence. According to the dependencies among different request types, this stage first combines different request types to construct various abstract request sequences. Then, this stage generates every parameter value for every request template in the request sequences. Finally, this stage generates concrete HTTP requests to test the target service.

Test case execution and monitoring. In this stage, concrete test requests in the form of sequences would be sent to the SUT, and the fuzzer would monitor the execution of test cases. According to the response of each test request, the fuzzer can analyze the result of test case execution. If a test request receives an error response request with the status code in the 40X range, it means that the request does not pass the first check of the target service for the syntax and semantics. This request is refused to execute by the SUT. Thus, it is invalid to test program deep logic. On the contrary, the target service would perform a behavior according to a valid test request. If the service behaves normally, the request will receive a response code in the 20X range. If a test request in the generated test case triggers a response status code in the 50X range, the fuzzer considers that an error is triggered and stores the test cases for future analysis.

Through the above process, the existing REST API fuzzers automatically generate test requests in the form of sequences based on OpenAPI specifications. These test cases are used to test an SUT via its RESTful APIs, and the fuzzers record the unique errors triggered in different states.

3.3. Large Language Models

Emerging pre-trained large language models (LLMs) have demonstrated impressive performance in understanding the grammar and natural language semantics [50]. Due to their logical analysis capabilities, an increasing number of studies utilize LLMs for security analysis [40]. LLMs, having been trained on a wide range of corpora, possess the ability to perform specific tasks without requiring additional training to handle certain security analysis tasks. Security personnel can leverage and control them through prompt engineering [51].

The capabilities of LLMs have various implications for RESTful APIs. RESTful APIs communicate through HTTP requests. LLMs, having been pre-trained on billions of internet samples, are equipped to understand the intricacies of the HTTP protocol. Consequently, LLMs can effectively parse and generate HTTP requests and responses, making LLMs valuable for tasks related to RESTful APIs.

4. Motivation

In this section, we provide an invalid test request from GitLab as an example to illustrate our addressed limitations and the motivation of our method. We firstly conclude two limitations of existing test case generation. Next, we present our motivation for optimizing test case generation and designing our method to revise the invalid test requests.

4.1. Limitations of Existing Test Case Generation

Existing RESTful API fuzzers struggle to generate appropriate parameter values for each request type and thus generate many invalid test requests that receive error responses with 40X status codes from the SUTs. The invalid requests are rejected by the SUTs and fail to exercise service functionality in depth, leading to poor fuzzing performance.

The invalid test requests are often caused by illegal values of partial request parameters. In other words, it is challenging for fuzzers to simultaneously generate valid values for all parameters. Figure 1a shows an original invalid test request and its error response when testing the WordPress *Pages* API. The request attempts to create a **page** resource but fails. Although most of the parameter values in the request are legal, a few incorrect parameter values (i.e., “date”, “date_gmt”, “menu_order” and “template”) result in failure. The existing RESTful API fuzzers discard these invalid tested requests and subsequently generate new test requests by re-instantiating all parameter values in this request type. Thus, the first limitation of existing fuzzers is as follows:

Limitation 1: Existing test case generators are unaware of errors in the invalid tested requests, resulting in subsequently generating more new invalid test requests that contain the same incorrect parameters.

Furthermore, the strategies of parameter value generation of the existing RESTful fuzzers find it hard to generate legal values for the incorrect parameters due to several reasons:

(1) The existing RESTful fuzzers are overly reliant on the OpenAPI specifications, whose completeness and accuracy cannot be guaranteed. As shown in Figure 1a, the incorrect parameters “date”, “date_gmt” and “template” have specific format constraints. However, they are only described as type *string* in the input OAS. Thus, the incomplete information makes it difficult to generate legal values for such parameters. Meanwhile, the parameter “menu_order” is described as type *string* in the specification, but according to the response, it should be an *integer*. Thus, the incorrect description leads to the generation of invalid values for it.

(2) Some parameter values depend on preceding resources related to the API. Without dynamic awareness of the SUT’s resource information, it is difficult to generate legal values for such parameters. For example, the incorrect parameters “author” and “parent” are dependent on pre-existing related resources, as shown in Figure 1b. Thus, the second limitation of existing fuzzers is as follows:

Limitation 2: The existing strategies of parameter value generation find it hard to generate legal values for these incorrect parameters due to the lack of effective information related to them. Only relying on the information extracted from the OAS is not enough to render legal values for them.

<pre> POST /wp/v2/pages HTTP/1.1 { "date": "fuzz032601", "date_gmt": "fuzz032601", "slug": "fuzz032601" "parent": 44, "author": 44, "featured_media": 44, "comment_status": "open", "ping_status": "open", "menu_order": "fuzz032601", "meta": {}, "template": "fuzz032601", } </pre>	<pre> POST /wp/v2/pages HTTP/1.1 { "date": "2023-03-26T01:00:00", "date_gmt": "2023-03-26T01:00:00", "slug": "fuzz032601" "parent": 44, "author": 44, "featured_media": 44, "comment_status": "open", "ping_status": "open", "menu_order": 0, "meta": {}, "template": "", } </pre>	Constructing a GET request: <pre> GET /wp/v2/pages HTTP/1.1 </pre>	<pre> POST /wp/v2/pages HTTP/1.1 { "date": "2023-03-26T01:00:00", "date_gmt": "2023-03-26T01:00:00", "slug": "fuzz032601" "parent": 1, "author": 1, "featured_media": 44, "comment_status": "open", "ping_status": "open", "menu_order": 0, "meta": {}, "template": "", } </pre>
<pre> HTTP/1.1 400 Bad Request { Invalid parameter(s): "date": "Invalid date.", "date_gmt": "Invalid date.", "menu_order": "menu_order is not of type integer.", "template": "template is not one of ." } </pre>	<pre> HTTP/1.1 400 Bad Request { Invalid author ID. Invalid post parent ID. } </pre>	Accessing the API-related resources: <pre> HTTP/1.1 200 OK [{"id": 20, "author": 11, "parent": 12, ...}, {"id": 23, "author": 22, "parent": 13, ...}] </pre>	<pre> HTTP/1.1 201 Created </pre>
(a)	(b)	(c)	(d)

Figure 1. Motivation example of revising an invalid tested request to generate new test requests. (a) An invalid tested request and its error response. (b) A new test request generated by revising incorrect parameters according to the semantic information in the error response and the new error response. (c) Constructing a GET request to access the API-related resource information containing the legitimate values for incorrect parameters from SUTs. (d) A new test request generated by revising incorrect parameters according to the API-related resource information and the new success response.

4.2. Dynamic Error Responses of Invalid Test Requests

We find that the dynamic error responses of invalid requests usually contain suggestive descriptions about which parameters are incorrect, why they are incorrect and how the legal parameter values should be. Guided by the dynamic error responses, we could revise the invalid tested requests to generate new test requests with a higher probability to be valid.

Specifically, the suggestive descriptions in the error responses often contain rich semantic information indicating the legal value format of the incorrect parameters. Therefore, we can easily revise the invalid parameter values by understanding the semantic information in the error response. For example, as shown in Figure 1a, the error response describes that there are 4 incorrect parameters. Specifically, “date” and “date_gmt” are invalid dates, and “menu_order” is not an integer. Thus, the semantic information in the error response can guide us to correct the parameters and generate a new test request for this request type, as shown in Figure 1b.

4.3. API-Related Resources of RESTful Service

Sometimes, the semantic information in the prompt description about incorrect parameters is not enough to generate valid values for them. As shown in Figure 1b, the description in error response about the incorrect parameters “author” and “parent” is merely “Invalid Value”.

We find that the existing API-related resources on the SUTs often contain legitimate parameter values. By actively accessing related resource information for each request type, we can extract legitimate candidate values for these parameters.

Following the guidelines on the CRUD semantics of HTTP methods [52], we can access the API-related resources by constructing and sending a GET request with the same endpoint path as the original request. For example, in Figure 1c, we construct a GET request to access the resources related to the invalid request, such as the list of all pages. Then, we extract the needed parameter values from the successful response request. Finally, we generate a new test request, as shown in Figure 1d, using the parameter values obtained from the resource information.

To actively access the API-related resource, we need to consider the resource dependency. The resource path is a string identifying a service resource and its parent hierarchy. Typically, the path is a (non-empty) sequence matching the regular expression:

$$(<resourceType>/<resourceIdentifier>/)^{+}$$

The *resourceType* denotes the type of a service resource, and *resourceIdentifier* is the specific identifier for the resource of that type. The last resource named in the path is typically the specific resource related to the request type. The hierarchy of the endpoint path captures the dependency among resources. For example, here is an endpoint path in WordPress *Posts* API: */posts/{post_id}/revisions/{revision_id}* that points to a specific **revision** resource of a **Post**. The **Revision** is a sub-resource that depends on its parent resource **Post**. In other words, accessing the **Revision** resource requires prior knowledge of its parent resource **Post**.

5. Method

DynER is an optimized test case generation for RESTful API fuzzers aiming to improve the effectiveness of test cases. Based on the invalid tested requests and their error response requests received from SUTs, DynER purposefully revises the errors of invalid tested requests to generate new test requests that are more likely to be valid ones.

DynER can be applied to a wide range of RESTful API fuzzers to improve fuzzing performance. We applied it to a state-of-the-art fuzzer RESTler and implemented DynER-RESTler. Figure 2 shows the framework of DynER-RESTler, i.e., an RESTful API fuzzer optimized with DynER.

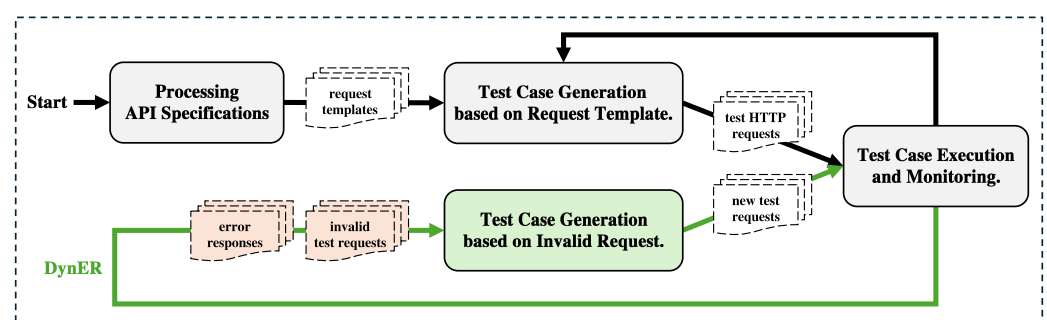


Figure 2. Framework of DynER-RESTler: the RESTful API fuzzer optimized with DynER.

The basic workflow of DynER-RESTler is as follows. Specifically, during the RESTful API fuzzing process, when the test case execution and monitoring module receives an error response with a 40X status code from SUT, it forwards the error response and the corresponding invalid test request to the DynER module. DynER then purposefully revises the invalid test request and generates a new, more probably valid test request. These new test requests are subsequently sent by fuzzer for testing.

In the following, we present the detailed design of DynER. First, we introduce the overall framework of DynER, i.e., the workflow of generating new test cases by revising

previous invalid test requests guided by the error responses (Section 5.1). Then, we introduce two innovative strategies for revising incorrect parameters (Sections 5.2 and 5.3).

5.1. Overview of DynER

Figure 3 shows the overview of DynER. It is a novel test case generation strategy that generates new test requests by revising the previous invalid tested requests guided by the corresponding error responses. The main process of the DynER is as follows.

The **input** of DynER consists of error responses and their corresponding invalid requests collected during the dynamic fuzzing process. Specifically, the baseline fuzzer automatically sends numerous randomly generated test requests to the target service. DynER monitors and captures the invalid requests that receive error responses from SUT (i.e., with the HTTP status code in the 40X or 50X range).

Extracting suggestive information from error responses. DynER extracts the status code and the suggestive description content in natural language form from the error response requests. These descriptions may provide concise details about the nature of the error, which include specific reasons for the failure, potential fixes or hints on what parameters might need revision.

Performing two strategies to revise the error parameters. Importantly, DynER designs two innovative strategies for revising the error parameters in the invalid tested requests, one based on prompting LLM to understand the semantic information of error responses, and the other based on recursively constructing parent requests to access the API-related resource for SUTs actively. DynER performs the two error parameter revision strategies to generate new value for the error parameters in the original invalid tested request and then generate new test requests. The detailed design of the two strategies is introduced in Sections 5.2 and 5.3.

Finally these new requests are tested during the fuzzing process.

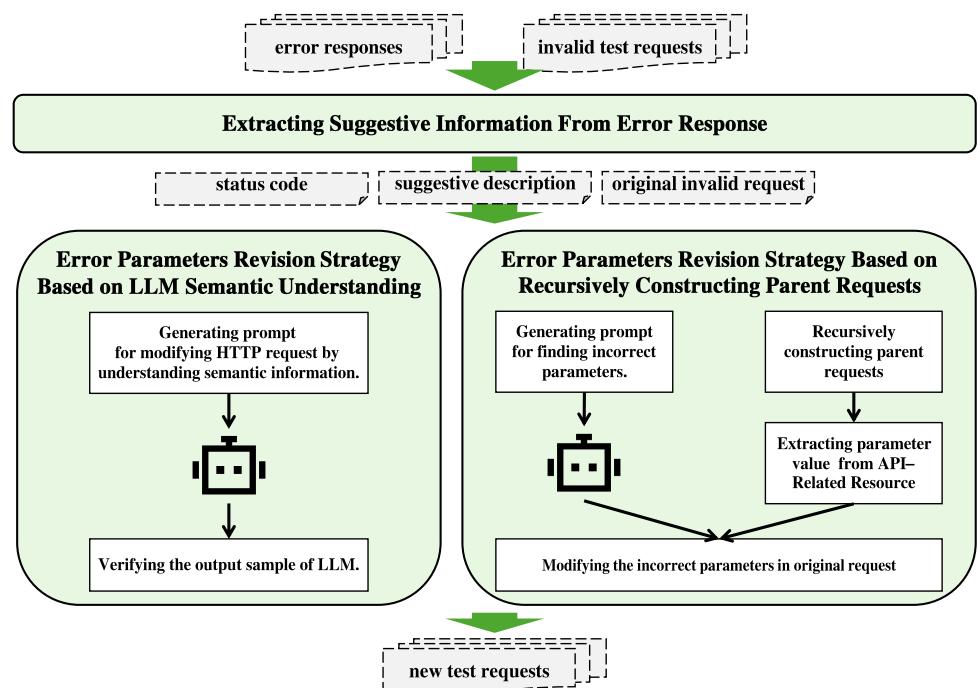


Figure 3. Framework of DynER.

5.2. Error Parameter Revision Strategy Based on LLM Semantic Understanding

As mentioned in Section 4.2, the suggestive description extracted from the error responses usually contains the details about the format and constraint of the incorrect parameters [53]. Guided by descriptive information, humans can manually generate valid

values for erroneous parameters and subsequently create new test requests. This process is easy for humans due to their ability to comprehend the semantic information in natural language descriptions.

However, automating this process during fuzz testing presents several difficulties. The suggestive description is conveyed in the form of natural language, lacking a standardized format. Moreover, the style of these natural language descriptions varies significantly across different target services, entirely dependent on the developers.

Natural language processing (NLP) [54] offers certain advantages in handling natural language description. However, to apply NLP to RESTful API fuzzing, we first need to summarize and categorize the types of semantic information found in error responses, such as “parameter type mismatch” and “parameter format error”. This requires a lot of manual effort. Additionally, due to the diverse implementation methods of suggestive information in error responses across various target applications, the scalability of this approach may be limited.

Inspired by the advanced semantic understanding capabilities of existing LLMs and their familiarity with the format of an HTTP request message, we propose an error parameter revision strategy based on LLM semantic understanding. This strategy leverages LLMs to interpret the semantic information conveyed in suggestive descriptions. Subsequently, based on the semantic understanding, the strategy revises the erroneous parameters in the original invalid requests and generates new test requests in the correct format.

Algorithm 1 shows the process of the error parameter revision strategy based on LLM semantic understanding. The strategy handles each invalid test request through several iterations until a valid request for this test request is achieved (line 1 sets the maximum number of iterations). The reasons for multiple iterations are as follows. (1) LLMs may not successfully revise all errors in an invalid request on the first attempt, and (2) sometimes, an error response only describes a subset of the incorrect parameters. Therefore, iterative revising and generating new requests and then sending them for testing, may obtain additional suggestive descriptions, allowing for cumulative revisions. However, if a new request does not result in new suggestive descriptions in its error response (line 13), the strategy concludes that the LLM is unable to revise the incorrect parameters in the invalid request, leading to termination. Additionally, we consider cases where invalid requests contain parameters that cannot be revised through semantic understanding. Therefore, if the number of iterations exceeds the set maximum limit (line 3), the strategy will also be terminated.

In each revising iteration (lines 2–20), the strategy first extracts the suggestive natural language description and the status code from the dynamic error response requests (line 8). Then, it generates specialized prompts according to the invalid request message *OriReq*, the error suggestive description *SuggDes* and the status codes *StatusCode* (line 9). These prompts are designed to guide the LLM in revising incorrect content within the invalid request and generating new test HTTP requests. The prompt template is shown in Figure 4. Subsequently, the strategy uses prompt engineering to guide the LLM in generating new test requests that are more likely to be valid based on the original invalid request (line 10). Despite its impressive capabilities, the LLM may occasionally produce incorrect or misleading outputs. Therefore, before the output of LLM is considered a usable test request, it undergoes rigorous verification (line 11). The used parameter *NewReq* is the output test request of LLM. Only those test requests that pass the verification process proceed to the fuzzing phase and are sent to the target service (line 12).

Algorithm 1 Error Parameter Revision Strategy Based on LLM Semantic Understanding

Input: *OriReq*: original invalid test request
OriRes: error response request
Output: *NewReq*: new test request
NewRes: new response request

```

1:  $MaxLLMTryTime \leftarrow 3$ 
2: while OriRes.statusCode  $\in$  4XX  $\parallel$  OriRes.statusCode  $\in$  5XX do
3:   if LLMtry == MaxLLMTryTime then
4:     break from while cycle (goto line 21)
5:   else
6:      $LLMtry \leftarrow LLMtry + 1$ 
7:   end if
8:   SuggDes, StatusCode  $\leftarrow$  ExtractInfo(OriRes)
9:   ReqRevisePrompt  $\leftarrow$  generatePrompt4Req(OriReq, SuggDes, StatusCode)
10:  NewReq  $\leftarrow$  LLMreviseReq(ReqRevisePrompt)
11:  if checkLLMReq(NewReq) then
12:    NewRes  $\leftarrow$  sendTest(OriReq)
13:    if NewRes == OriRes then
14:      break from line2
15:    else
16:      OriReq  $\leftarrow$  NewReq
17:      OriRes  $\leftarrow$  NewRes
18:    end if
19:  end if
20: end while
21: return OriReq, OriRes

```

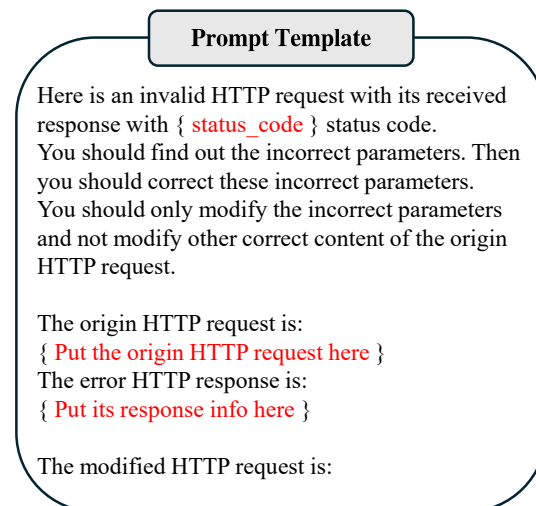


Figure 4. Prompt template for revising incorrect content of an invalid request.

Verifying the output sample of LLM. The strategy verifies the output of LLM from the following perspectives:

1. *Endpoint path.* The LLM should revise only the parameter fields within the endpoint path of the original request. If the fixed fields of the path are revised, i.e., the endpoint is revised, the resulting request no longer corresponds to the original request and becomes unusable. To prevent this, the strategy establishes multiple path-checking rules as follows:
 - The number of path separators, i.e., “/”, should remain consistent before and after.
 - Any two consecutive parts separated by “/” should not be modified simultaneously.

2. *HTTP method.* The LLM should not revise the HTTP method of the original request. If the method type is revised, the output will fail verification and will be discarded.
3. *User authentication information.* Modifying the test user authentication information could affect the results of the test request. Therefore, during verification, the authentication information of output requests is restored to the original test user authentication details.

5.3. Error Parameter Revision Strategy Based on Recursively Constructing Parent Requests

In the previous section, DynER designs a highly effective error parameter revision strategy when the suggestive description contains rich semantic information. However, this strategy will lose effectiveness when the semantic information is too scarce to guide the generation of legitimate values for the error parameters. Therefore, a strategy is required to ensure efficient test case generation in such situations.

Based on the design principles of HTTP CRUD semantics and the resource design characteristics of RESTful APIs, we propose an error parameter revision strategy based on recursively constructing parent requests. Specifically, the strategy accesses the related resource by automatically constructing and sending a GET request with the same endpoint path as the original invalid request. For example, Figure 1c shows a constructed GET request to access resources related to the invalid request in Figure 1b. Then, it extracts legitimate values for the error parameters from the successful response. These values are then used to modify the error parameters, generating new test requests for fuzzing.

Significantly, there is a situation where the resource related to the current invalid request may depend on its parent resources, that is, multi-level resource dependencies exist. This will cause the failure of the GET request to access the related resource (bottom-level) due to the lack of correct resource ID parameter assignment, which is usually dependent on parent resources. For example, here is an invalid tested request for the request type *POST /posts/{post_id}/revisions/{revision_id}*. Directly constructing the GET request under the endpoint to access a specific **Revision** resource may fail due to the incorrect parameter values “revision_id” and “post_id”. The two parameters are dependent on the multi-level parent resources, i.e., the existing **Revision** resource and **Post** resource.

To address this issue, the strategy leverages the resource dependencies implicit in the endpoint *Path* to recursively construct *parent requests* with progressively shorter paths, thereby accessing multi-level parent resources. By shortening the *Path* step by step, we continue constructing parent requests from the bottom up until the relevant parent resource is obtained, which we refer to as **the top-level parent resources**. For example, to construct parent requests for *POST /posts/{post_id}/revisions/{revision_id}*, we might progressively create requests such as *GET /posts/{invalid_post_id}/revisions/{invalid_revision_id}*, *GET /posts/{invalid_post_id}/revisions*, *GET /posts/{invalid_post_id}* and *GET /posts*.

Based on the top-level parent resources accessed, valid resource ID parameter values can then be obtained. For example, the valid “post_id” parameter values can be retrieved from the successful response of *GET /posts*. Subsequently, using the ID values of the higher-level parent resources, we recursively construct child requests from the top down until the relevant API-related resources are accessed. For instance, we might create requests *GET /posts/{valid_post_id}/revisions* and *GET /posts/{valid_post_id}/revisions/{valid_revision_id}*. Finally, these valid candidate values are used to revise the incorrect parameters in the original invalid request.

Algorithm 2 outlines the process of the error parameter revision strategy based on recursively constructing parent requests. Firstly, this strategy determines the parameters that caused the invalid request. Specifically, for each invalid test request (line 1), the strategy extracts the suggestive natural language description and the status code from the dynamic error response (line 2). Then, the strategy guides the LLM in determining the incorrect parameters of the invalid request through prompt engineering and outputs the list of incorrect parameter names (lines 3–4). The prompt template is shown in Figure 5.

Algorithm 2 Error Parameter Revision Strategy Based on Recursively Constructing Parent Requests

Input: *OriReq*: original invalid test request

OriRes: error response request

Output: *NewReq*: new test request

NewRes: new response request

```

1: if OriRes.statusCode ∈ 4XX then
2:   SuggDes, StatusCode ← ExtractInfo(OriRes)
3:   ParamFindPrompt ← generatePrompt4Param(OriReq, SuggDes, StatusCode)
4:   ErrorParams ← LLMFindErrorParams(ParamFindPrompt)
5:   ResourceReqs, ResourceResps ← constructParentReqs(OriReq)
6:   ParamValues ← extractResourceParams(ErrorParams, ResourceReqs, ResourceResps)
7:   while OriRes.statusCode ∈ 4XX do
8:     CurValueSet ← combineParamsEachValue(ParamValues)
9:     NewReq ← reviseReqParams(ErrorParams, CurValueSet)
10:    for each ErrorP ∈ ErrorParams do
11:      if ErrorP ∉ CurValueSet then
12:        NewReq ← delParam(ErrorP)
13:      end if
14:    end for
15:    NewRes ← sendTest(NewReq)
16:    if NewRes.statusCode ∈ 4XX and NewRes ≠ OriRes then
17:      goto Algorithm1
18:    else
19:      OriReq ← NewReq
20:      OriRes ← NewRes
21:    end if
22:  end while
23: end if
24: return OriReq, OriRes

```

Prompt Template

Here is an invalid HTTP request with its received response with { **status_code** } status code. You should find out the incorrect parameters. Then return these parameter names in the format of list. The list could be parsed by JSON.

The origin HTTP request is:
 { **Put the origin HTTP request here** }
 The error HTTP response is:
 { **Put its response info here** }

Figure 5. Prompt template for finding out the incorrect parameters of an invalid request.

Secondly, the strategy accesses the related resources, which have the same endpoint path as the original invalid request, by recursively constructing parent requests (line 5). Subsequently, the strategy extracts legitimate values for the error parameters from the successful responses of the related resource requests (line 6).

Finally, the strategy revises the invalid request in several iterations until generating a valid test request (lines 7–22). In each iteration, the strategy randomly selects a value for each error parameter (line 8) to replace its original error value (line 9). If the related resource response does not contain the candidate value for the error parameters, the strategy deletes them (lines 10–14). Then, the new test request enters the fuzzing process and is sent to the target service (line 15). If the latest request receives a different response in the 40X range, it

is handled by the error parameter revision strategy based on LLM semantic understanding (lines 16–17).

6. Evaluation

To evaluate the performance of DynER, we apply DynER to the state-of-the-art fuzzer RESTler and implement DynER-RESTler. We aim to answer the following research questions with the evaluation:

- **RQ1:** How is the improvement of DynER in generating effective test cases for the most representative RESTful API fuzzer RESTler?
- **RQ2:** How is the fuzzing performance of DynER-RESTler compared to the existing tools, such as RESTler and foREST?
- **RQ3:** How do the two error parameter revision strategies of DynER contribute to its overall performance?

6.1. Implementation and Experimental Setting

We apply DynER to the state-of-the-art fuzzer RESTler and implement an optimized fuzzer named DynER-RESTler. We implement the prototype DynER for the proposed optimized test case generation with Python 3.8. DynER contains 2K+ lines of Python code. We conduct our evaluation experiment in a local network environment. Note that all compared tools were run in the same environment for fairness.

Compared fuzzers. To evaluate the improvement that DynER could contribute to RESTful API fuzzers, we apply DynER to the state-of-the-art fuzzer RESTler and implement an optimized fuzzer named DynER-RESTler. RESTler is a well-established tool in the RESTful API fuzzing field and, in fact, the most popular REST API testing tool in terms of GitHub stars. Moreover, RESTler is commonly used as the baseline fuzzer in the previous studies [7,9,10]. To further evaluate the fuzzing performance of DynER-RESTler compared to the existing tools, we select a recently open-sourced state-of-the-art fuzzer foREST. foREST statistically outperforms other existing tools in their experiments, such as RESTler and EvoMaster. We do not compare with other fuzzers due to their poor performance compared to foREST or issues with reproducibility.

RESTful Service Selection. As shown in Table 1, we evaluate the above fuzzers on WordPress and GitLab via 15 RESTful APIs, and the reasons for selecting these targets are as follows. First, GitLab and WordPress are widely used RESTful services. Evaluating their security is meaningful for vendors and users. Second, the request types in the RESTful APIs of GitLab contain many parameters to support a wide variety of functionalities. On the contrary, the request types of WordPress are relatively simple ones with fewer parameters, which limits the difficulty in generating test cases. Thus, in the evaluation, we can analyze the performance of fuzzers on the SUTs with different functional complexity. Meanwhile, they are commonly used experimental objectives in previous studies [5,9,10].

Evaluation metrics. To measure the performance of each fuzzer on effective test case generation, we evaluate the fuzzers with the following four metrics.

- **Code Coverage (LOCs).** Code coverage can reflect the exploration capability of the fuzzers. We measure the number of covered code lines triggered by test requests.
- **Number of successfully tested request types (STRTs).** The more unique request types are successfully tested, the more kinds of behaviors are triggered by fuzzers. We measure the number of successfully requested unique request types that obtain responses in the 20X range, which can reflect the tested depth of code logic of the RESTful service.
- **Pass rate of test requests(PRTT).** Receiving successful responses means the target service executes the test request, which passes the syntax and semantic checking. We measure the pass rate of each fuzzer to the syntax and semantic checking of a web service, which is calculated by dividing the number of successful responses in the 20X and 50X range by the number of total responses.

- **Number of unique errors(Bugs).** In the context of RESTful service, an *error* is considered to trigger the response in 50X range. A bug can be related to many *errors*. We manually classified the *errors* into unique bugs according to response bodies, server logs, etc.

Table 1. The SUTs and 15 RESTful APIs used in the experiments.

RESTful Service	API Group	# Request Types	Description
WordPress5.8.1	Posts	11	API to maintain <i>posts</i> resource.
	Pages	11	API to maintain <i>page</i> resource.
	Users	8	API to maintain <i>user</i> resource.
	Medias	5	API to maintain <i>media</i> resource.
	Categories	5	API to maintain <i>category</i> resource.
	Tags	5	API to maintain <i>tag</i> resource.
	Comments	5	API to maintain <i>comment</i> resource.
	Themes	2	API to maintain <i>theme</i> resource.
	Taxonomies	2	API to maintain <i>taxonomy</i> resource.
	Types	2	API to maintain <i>type</i> resource.
	Status	2	API to maintain <i>status</i> resource.
	Settings	2	API to maintain <i>setting</i> of the website.
GitLab14.4.2-ee.0.	Projects	33	API to maintain <i>project</i> resource.
	Issues	24	API to maintain <i>issue</i> resource.
	Groups	23	API to maintain <i>group</i> resource.

6.2. Effectiveness in Test Case Generation (RQ1)

To evaluate the improvement of DynER in generating effective test cases for RESTful API fuzzer, we conducted fuzz testing on three APIs for WordPress and three APIs for GitLab using both RESTler and DynER-RESTler. We focus on the gap in the ability to generate high-quality test requests and valid test cases between the two tools. The gap represents the improvement contributed by DynER.

We evaluated RESTler and DynER-RESTler with their bug checker modules turned off to exclude the bug checker's request data from interfering with the evaluation results. Each fuzzing test lasted for 2 h. Both RESTler and DynER-RESTler were configured with the same simple fuzzing dictionary and the OpenAPI specification (OAS) provided by a third party to ensure the fairness of the experiment. The detailed experimental results are shown in Table 2.

Table 2. The improvement in effectiveness of test cases that DynER contributes to RESTler.

RESTful Service	API Group	#Request Types	RESTler				DynER-RESTler			
			LOCs	STRs	PRTT	Bugs	LOCs	STRs	PRTT	Bugs
WordPress	Pages	11	13,542	0	<1%	0	16,931	10	40.90%	0
	Posts	11	13,774	0	<1%	0	18,404	10	58.29%	0
	Media	5	13,618	0	<1%	0	16,834	4	73.98%	1
GitLab	Projects	33	2164	5	7.14%	0	7669	29	32.91%	3
	Issues	24	1480	0	<1%	0	5799	22	36.87%	2
	Groups	23	1233	0	<1%	0	6429	21	45.36%	2

Coverage. As shown in Table 2, DynER significantly improves code coverage and successfully tests more unique request types. DynER-RESTler successfully tested over 90% of request types in 4/6 target APIs and over 80% of request types in all target APIs. Here, the percentages are calculated as STRs / the total number of request types for each API Group. Correspondingly, DynER-RESTler achieved higher code coverage than RESTler in all APIs. For instance, on the three WordPress APIs, code coverage increased by 25.03%, 33.61% and 23.62%, respectively, calculated as (LOCs of DynER-RESTler-LOCs

of RESTler)/LOCs of RESTler. The improvement was even more significant on GitLab APIs, with code coverage increasing by more than 200% across all three API groups. These results demonstrate that DynER effectively generates more valid test cases, leading to more comprehensive testing of target APIs.

Note that under the same experimental configuration and environment, the results of RESTler were poor. However, we conducted the experiment using the default configuration and repeated it multiple times, consistently obtaining nearly identical results.

Pass rate of test requests. DynER-RESTler generates more valid test requests than RESTler. For instance, the pass rate of test requests has increased to 73.98%, 40.90% and 58.29% on the three WordPress APIs, respectively. The average pass rate of DynER-RESTler on GitLab is 38.38%. This indicates that with fewer but higher-quality test requests, DynER-RESTler can explore target APIs more efficiently and thoroughly. In contrast, the pass rate of test requests generated by RESTler was lower than 1% in 5 out of 6 target APIs. DynER significantly improves the pass rate of test requests.

Bugs. Although DynER is not optimized for checkers that trigger specific errors, DynER-RESTler still detects more unique errors than RESTler. This is because the high-quality test cases generated by DynER-RESTler can deeply explore the service logic, thereby uncovering deep logic errors. Furthermore, DynER-RESTler has found three new bugs from GitLab and WordPress. We will elaborate on them more in Section 6.5.

6.3. Fuzzing Performance (RQ2)

To evaluate the overall fuzzing performance of DynER-RESTler. We apply RESTler, foREST and DynER-RESTler to fuzz two open source RESTful services, WordPress and GitLab. We evaluate the overall fuzzing performance via all API groups of each SUT, i.e., 12 APIs of WordPress and 3 APIs of GitLab. Each API group interacts with different kinds of resources on the service. Fuzzing via all the API groups of each SUT, the evaluated fuzzers can execute different code lines and explore different states of the service, which can examine overall fuzzing performance more comprehensively. Each fuzz test on each service lasts for 3 h.

Table 3 presents the detailed experimental results of the three fuzzers on four metrics. The overall fuzzing performance of RESTler is relatively poor, while foREST achieves significantly better performance than RESTler. However, when we apply DynER to RESTler, the overall fuzzing performance is significantly improved and even surpasses foREST in terms of code coverage, number of successfully tested request types and pass rate of test requests. The number of bugs discovered by DynER-RESTler also significantly increases.

Table 3. The fuzzing performance of RESTler, foREST and DynER-RESTler on WordPress and GitLab.

RESTful Service	API Group	#Request Types	RESTler				foREST				DynER-RESTler			
			LOCs	STRIs	PRIT	Bugs	LOCs	STRIs	PRIT	Bugs	LOCs	STRIs	PRIT	Bugs
WordPress	All	60	17,699	4	<1%	2	21,933	48	6.50%	9	22,351	54	47.71%	7
GitLab	All	80	2255	0	<1%	2	15,989	57	8.57%	5	16,267	70	34.90%	4

The average pass rate of DynER-RESTler is 41.21% and 26.33% higher than foREST on WordPress and GitLab, respectively. This means that DynER-RESTler can generate high-quality requests, successfully testing more unique request types than foREST. Specifically, the average number of unique request templates covered by DynER-RESTler is 12.5% higher on WordPress and 22.80% higher on GitLab compared to foREST.

Due to the higher pass rate of generated test requests, DynER-RESTler achieves higher code coverage, which covered 418 more lines on WordPress and 278 more lines on GitLab than foREST. To further analyze the effectiveness of generated test cases, we inspect the growth of code coverage over the number of sent test requests and over time, respectively, when fuzzing WordPress. As shown in Figure 6, the experimental results demonstrate that DynER-RESTler consistently achieves higher code coverage compared to foREST when the same number of generated test requests are sent to SUT. As shown in Figure 7, foREST

outperforms DynER-RESTler in code coverage only within the first 30 min of testing. We find foREST sends a greater number of test requests within the same time. Even though the proportion of valid requests was not high, the code coverage temporarily increases due to invalid requests triggering some extra less important code. Beyond the initial period, DynER-RESTler consistently maintains superior code coverage. The experimental results demonstrate that more valid test requests can exercise the code logic of SUTs more thoroughly, thereby DynER-RESTler achieves higher code coverage finally.

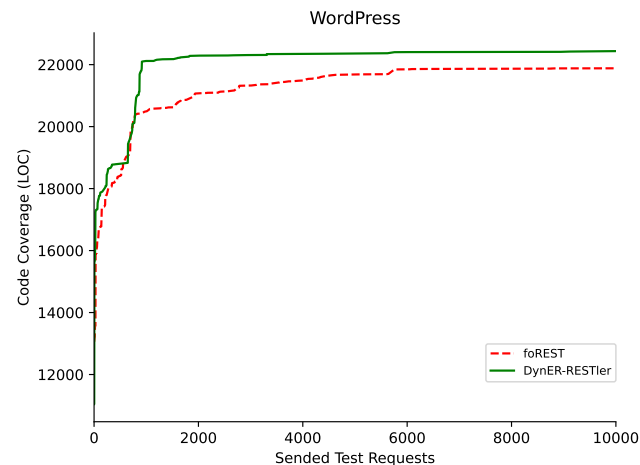


Figure 6. The code coverage over time when fuzzing WordPress with DynER-RESTler and foREST, respectively.

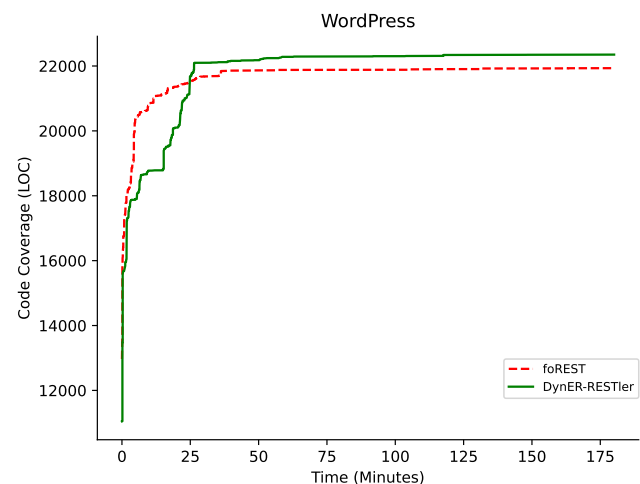


Figure 7. The code coverage over the number of sent test requests when fuzzing WordPress with DynER-RESTler and foREST, respectively.

DynER-RESTler detects more unique errors than RESTler. Specifically, it detected five more bugs in WordPress and two more bugs in GitLab compared to Restler. foREST detected more errors than DynER-RESTler. Through deeper analysis, we find that foREST detects these bugs due to its extra designs in triggering specific errors, such as using special UTF-8 strings and characters (e.g., “:”). DynER-RESTler is able to successfully test the request types that trigger these errors. Due to DynER-RESTler utilizing default error triggering mechanisms of RESTler without the extra designs, these bugs are missed. Therefore, with the extra designs in triggering specific errors, DynER-RESTler would detect these bugs. It is worth emphasizing that DynER focuses on exploring deeper logic rather than on the specific mechanisms for triggering individual bugs. DynER-RESTler still discovers three new bugs in GitLab that foREST could not detect. We discuss them more in Section 6.5.

6.4. Ablation Study (RQ3)

We conduct an ablation study on each strategy to investigate how the two error parameter revision strategies contribute to enhancing the effectiveness of generated test cases. Specifically, we construct **RESTler+Semantic** by applying the error parameter revision strategy based on LLM semantic understanding and **RESTler+Resource** by applying the error parameter revision strategy based on recursively constructing parent requests. By comparing the performance of RESTler, RESTler+Semantic, RESTler+Resource and DynER-RESTler, we can evaluate the contribution of different error parameter revision strategies to the pass rate and code coverage for test cases. We select the API group with most request types in WordPress and GitLab. We conduct the ablation study with the four fuzzers on the two APIs. Each fuzz test last for 2 h. The experimental results are shown in Table 4, from which we can draw the following conclusions.

Table 4. The code coverage, successfully tested request types and pass rate when fuzzing with RESTler, RESTler+Semantics, RESTler+Resource and DynER-RESTler respectively.

RESTful Service	API Group	#Request Types	RESTler			RESTler+Semantics			RESTler+Resource			DynER-RESTler		
			LOCs	STRIs	PRTT	LOCs	STRIs	PRTT	LOCs	STRIs	PRTT	LOCs	STRIs	PRTT
WordPress	Posts	11	13,774	0	<1%	17,179	5	84.54%	18,101	9	67.31%	18,397	10	50.26%
GitLab	Projects	33	2164	5	7.14%	6389	20	39.75%	7777	27	30.62%	7669	29	32.91%

Both the two strategies contribute to boosting the effectiveness of generating test cases. DynER-RESTler performs better than RESTler+Semantic and RESTler+Resource on all APIs in successfully tested request types. Meanwhile, both RESTler+Semantic and RESTler+Resource perform better than RESTler. For instance, the number of request types covered by DynER-RESTler is 6.06% larger than RESTler+Resource, 27.27% larger than RESTler+Semantic and 72.73% larger than RESTler on GitLab Projects API.

Similarly, the code coverage of RESTler+Resource and RESTler+Semantic is higher than RESTler on WordPress Posts API. Figures 8 and 9 show the growth of the code coverage for RESTler, RESTler+Semantics, RESTler+Resource and DynER-RESTler, respectively, over the number of tested requests and over testing time when fuzzing WordPress Post API. The experimental results show that DynER-RESTler consistently achieves the best results, demonstrating that DynER-RESTler is the most effective in generating valid test cases.

Notably, RESTler+Resource outperforms the other three fuzzers on the GitLab Projects API in term of code coverage. Further analysis reveals that GitLab provides limited semantic information in their error response and the API relies heavily on resource dependencies. As a result, the error parameter revision strategy by recursively constructing parent requests to actively access API resources proves to be more effective. However, by combining the two strategies, DynER-RESTler successfully tested more request types. In other words, DynER-RESTler achieved the best fuzzing performance.

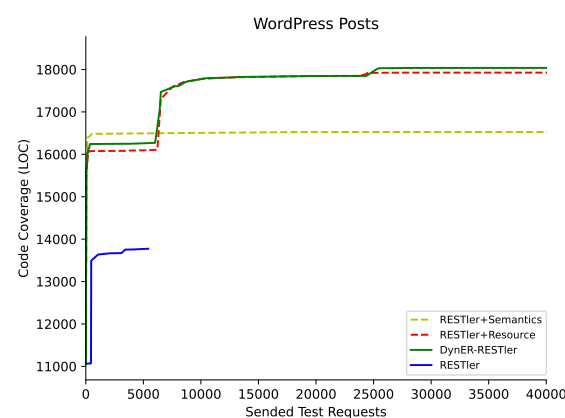


Figure 8. The code coverage for RESTler, RESTler+Semantics, RESTler+Resource and DynER-RESTler, respectively, over the number of tested requests during fuzzing.

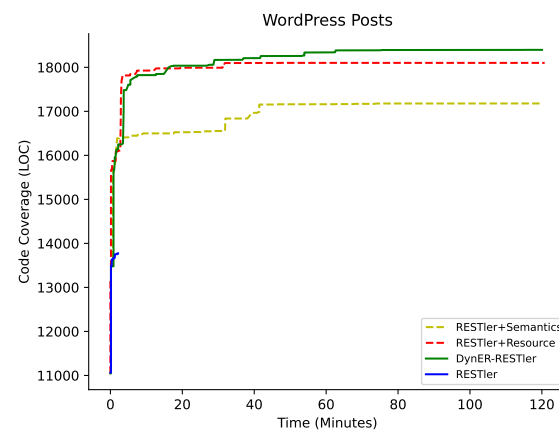


Figure 9. The code coverage of RESTler, RESTler+Semantics, RESTler+Resource and DynER-RESTler, respectively, over time during fuzzing.

6.5. Case Study of New Bugs

According to the experimental results, DynER-RESTler discovered two new bugs in GitLab and one new bug in WordPress. All these bugs are reproducible.

Figure 10 shows the bug in the request type “POST wp/v2/media” in WordPress. To trigger this bug, an HTTP POST request with a “multipart/form-data” message format is required. The existing fuzzers fail to trigger the bug because they are not aware of the error in the original message format of “application/json”. Thus, the fuzzers generate many invalid test requests with the incorrect message format of “application/json”. Based on the capabilities of LLMs that are proficient in the HTTP protocol, DynER-RESTler is able to generate the valid test requests with the correct “multipart/form-data” message format.

```

POST wp/v2/media
Content-Type: multipart/form-data; boundary=some-boundary
.....

--some-boundary
Content-Disposition: form-data; name="file";
filename="fuzz041801"
Content-Type: application/octet-stream

(file content here)
--some-boundary--

```

```

HTTP/1.1 500 Internal Server Error
Sorry, this file type is not permitted for security reasons.

```

Figure 10. A new interesting bug detected in WordPress *Media* API. The test request triggering the bug should be in the message format of “multipart/form-data”.

Figure 11 shows the two bugs from GitLab. To trigger the first bug in the request type “PUT /api/v4/projects/{project_iid}/issues/{issue_iid}/reorder”, the two parameters “project_iid” and “issue_iid” must be the corresponding *existing resource IDs*. DynER-RESTler actively accesses the API-related resources and obtains these IDs. Thus, it could generate the valid test requests. The second bug in the request type “POST /api/v4/projects/user/{user_id}” is caused by the parameter “use_custom_template”. The bug is difficult to trigger because this request type includes many other parameters with format constraints. If any other parameter values are incorrect, the bug will not be triggered. DynER could dynamically revise other incorrect parameters and trigger this bug successfully.

```
PUT /api/v4/projects/{project_iid}/issues/{issue_iid}/reorder
.....
```

```
{“move_after_id”: 2, “move_before_id”: 2}
```

```
HTTP/1.1 500 Internal Server Error
```

```
POST /api/v4/projects/user/{user_id}?name={project_name}
.....
```

```
{“use_custom_template”: true}
```

```
HTTP/1.1 500 Internal Server Error
```

Figure 11. Two new interesting bugs detected in GitLab *Issues* API and *Projects* API. The test request triggering the first bug should contain the two parameters “project_iid” and “issue_iid”, whose value must be the corresponding *existing resource IDs*. The second bug is caused by the “use_custom_template” parameter. The test request triggering this bug must assign a valid value for other parameters with format constraints.

In summary, DynER-RESTler generates high-quality test requests and significantly improves the effectiveness of generated test cases. Consequently, DynER-RESTler achieves better fuzzing performance by exploring the deep logic of systems under test (SUTs). Finally, DynER-RESTler still triggers new bugs without any extra design for triggering specific errors.

7. Discussion and Future Work

The experimental results show that our novel method DynER presented in this paper can improve the performance of RESTful API fuzzers significantly in terms of code coverage, request type coverage, pass rate of test requests and the number of detected bugs. Not only that, the fuzzing performance of our prototype tool DynER-RESTler significantly outperforms the state-of-the-art fuzzer foREST. This is contributed to by the innovative design of DynER that generates new test requests by purposefully revising the errors of invalid tested requests guided by dynamic error responses and the two new strategies for generating legal parameter values, which can enhance the weaknesses of existing strategies. However, we still need to consider the threats to validity, the limitations of our method and future work.

7.1. Threats to Validity

As all empirical evaluations, our study is subject to both external and internal threats to validity.

External validity may be impacted by the selection of RESTful services and the limited number of services tested, which can affect the generalizability of our method. To mitigate this threat, we select two widely used real-world services and included 15 APIs for testing. The rationale behind this choice is as follows: First, WordPress and GitLab are widely used RESTful services, and evaluating their security holds significant value for both vendors and users. These services are also frequently used as benchmarks in the previous research [5,8–10]. Second, GitLab’s RESTful APIs have a wide range of request types with numerous parameters supporting diverse functionalities. In contrast, WordPress has relatively simpler request types with fewer parameters, allowing us to analyze the performance of fuzzers across services with varying functional complexity. Third, we chose the open-source RESTful services so that we could conduct an in-depth analysis of the experimental results, including scrutinizing code coverage and categorizing detected failures into unique bugs. Although we have taken steps to ensure that the selected target

RESTful services are sufficiently representative, we plan to expand to a larger and more diverse set of services in future work.

Internal validity is influenced by tool implementation and configuration. To minimize these threats, we use the latest versions of RESTler and foREST, running them with their default (recommended) configurations. As testing tools may exhibit randomness in their results, we address this by running each tool five times and averaging the results. Additionally, the experimental results in this paper may be influenced by the specific implementation of our DynER approach. We have thoroughly reviewed our code to prevent any potential errors.

7.2. Future Work

DynER is a generic scheme that can be applied to a wide range of RESTful API fuzzers. In this paper, we apply DynER to RESTler and evaluate its effectiveness in improving the test case generation and fuzzing performance for RESTler. However, limited by RESTler's inherent capabilities, DynER's potential has not been fully explored, such as optimizing RESTful API vulnerability discovery tools. In the future, DynER can be applied to more fuzzers to optimize their test case generation and improve the fuzzing performance and the ability of vulnerability discovery.

DynER leverages LLMs to understand the semantic information of error responses and generate valid values with the simple zero-shot prompting. We will try more advanced prompt engineering techniques, such as chain-of-thought prompting, to improve the generation of valid values. Additionally, we will further explore the ability of LLMs in other tasks, such as generating non-trivial parameter values and inferring the constraints of parameters according to the characteristics of valid and invalid values.

Enhancing the bug detection capability of the RESTful API fuzzer is one of our next research objectives. DynER will contribute significantly to bug detection in our future work. For instance, if we apply a catalog of mutation operators to the test requests with the aim of violating security rules, DynER could help eliminate the influence of irrelevant error parameters by generating valid value for them. Then the test requests with the injected malicious data could be successfully executed by SUTs. Otherwise, the irrelevant error parameters would hinder the bug detection.

8. Conclusions

RESTful APIs are widely used in modern web and cloud services. RESTful API fuzzing plays an important role in ensuring the reliability of RESTful APIs by automatically generating test cases and exercising them with the SUTs. However, existing fuzzers still face challenges in test case generation, which severely hinders fuzzing performance.

In this paper, we propose DynER, a novel test case generation method for RESTful API fuzzers, guided by dynamic error responses during fuzzing to address these limitations. Based on the invalid test requests and the error responses received from the SUTs, DynER purposefully revises the errors in invalid test requests to generate new test requests during fuzzing. DynER incorporates two novel strategies for generating valid values for incorrect parameters by leveraging LLMs to understand the semantic information of error responses and actively accessing API-related resources on the SUT, respectively. We apply DynER to the well-known RESTful API fuzzer, RESTler and implement DynER-RESTler. We evaluate the improvements in the effectiveness of test cases and fuzzing performance contributed by DynER. The experimental results demonstrate that DynER significantly optimizes test case generation in terms of code coverage, request type coverage, pass rate of test requests and the number of detected bugs. Our evaluation further shows that DynER-RESTler significantly outperforms the state-of-the-art fuzzer foREST, achieving 41.21% and 26.33% higher average pass rates of test requests and 12.50% and 22.80% higher average numbers of unique request types successfully tested, respectively. Additionally, DynER-RESTler finds three previously unknown bugs.

In the future, to study the generalization of DynER, we plan to conduct additional experiments on more target services. To further explore and enhance DynER's potential, we will apply it to more RESTful API fuzzers and experiment with more advanced prompt engineering techniques. Additionally, we will explore the ability of LLMs in other tasks of RESTful API fuzzing, such as generating non-trivial parameter values and inferring the constraints of parameters based on the characteristics of valid and invalid values.

Author Contributions: Conceptualization, J.C., Z.P. and M.Z.; methodology, J.C. and Y.C. (Yuanchao Chen); software, J.C.; validation, Y.C. (Yuanchao Chen); Y.C. (Yu Chen) and Y.S.; formal analysis, J.C. and Y.L. (Yuwei Li); investigation, J.C. and Y.C. (Yu Chen); resources, Z.P., Y.L. (Yang Li) and Y.S.; writing—original draft preparation, J.C.; writing—review and editing, Y.C. (Yuanchao Chen), Z.P., Y.C. (Yu Chen) and Y.L. (Yuwei Li); visualization, J.C., Y.L. (Yang Li) and Y.S.; supervision, Z.P. and M.Z.; funding acquisition, Z.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the NSFC under No. 62202484.

Data Availability Statement: <https://github.com/starrychen1122/DynER>, accessed on 28 August 2024.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Gamez-Diaz, A.; Fernandez, P.; Ruiz-Cortes, A. An analysis of RESTful APIs offerings in the industry. In Proceedings of the International Conference on Service-Oriented Computing, Malaga, Spain, 13–16 November 2017; Springer: Cham, Switzerland, 2017; pp. 589–604.
- Google Inc. Available online: <https://www.google.com/> (accessed on 1 July 2024).
- Amazon Inc. Available online: <https://www.amazon.com/> (accessed on 1 July 2024).
- Golmohammadi, A.; Zhang, M.; Arcuri, A. Testing restful apis: A survey. *ACM Trans. Softw. Eng. Methodol.* **2023**, *33*, 1–41. [CrossRef]
- Atlidakis, V.; Godefroid, P.; Polishchuk, M. Restler: Stateful rest api fuzzing. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 25–31 May 2019; pp. 748–758.
- Viglianisi, E.; Dallago, M.; Ceccato, M. Resttestgen: Automated black-box testing of restful apis. In Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 23–27 March 2020; pp. 142–152.
- Liu, Y.; Li, Y.; Deng, G.; Liu, Y.; Wan, R.; Wu, R.; Ji, D.; Xu, S.; Bao, M. Morest: Model-based RESTful API testing with execution feedback. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 25–27 May 2022; pp. 1406–1417.
- Lin, J.; Li, T.; Chen, Y.; Wei, G.; Lin, J.; Zhang, S.; Xu, H. foREST: A Tree-based Black-box Fuzzing Approach for RESTful APIs. In Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 9–12 October 2023; pp. 695–705.
- Lyu, C.; Xu, J.; Ji, S.; Zhang, X.; Wang, Q.; Zhao, B.; Pan, G.; Cao, W.; Chen, P.; Beyah, R. {MINER}: A Hybrid {Data-Driven} Approach for {REST}{API} Fuzzing. In Proceedings of the 32nd USENIX Security Symposium (USENIX Security 23), Anaheim, CA, USA, 9–11 August 2023; pp. 4517–4534.
- Wu, H.; Xu, L.; Niu, X.; Nie, C. Combinatorial testing of restful apis. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 25–27 May 2022; pp. 426–437.
- Martin-Lopez, A.; Segura, S.; Ruiz-Cortés, A. RESTest: Black-Box Constraint-Based Testing of RESTful Web APIs. In Proceedings of the Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, 14–17 December 2020; Proceedings; Springer: Berlin/Heidelberg, Germany, 2020; pp. 459–475. [CrossRef]
- Kim, M.; Corradini, D.; Sinha, S.; Orso, A.; Pasqua, M.; Tzoref-Brill, R.; Ceccato, M. Enhancing REST API Testing with NLP Techniques. In Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, 17–21 July 2023; Association for Computing Machinery: New York, NY, USA, 2023; pp. 1232–1243. [CrossRef]
- Alonso, J.C.; Martin-Lopez, A.; Segura, S.; García, J.M.; Ruiz-Cortés, A. ARTE: Automated Generation of Realistic Test Inputs for Web APIs. *IEEE Trans. Softw. Eng.* **2023**, *49*, 348–363. [CrossRef]
- OpenAPI Specification. Available online: <https://swagger.io/specification/> (accessed on 1 July 2024).
- The Most-Comprehensive AI-Powered DevSecOps Platform | GitLab. Available online: <https://about.gitlab.com/> (accessed on 1 July 2024).
- Blog Tool, Publishing Platform, and CMS—WordPress.org. Available online: <https://wordpress.org/> (accessed on 1 July 2024).

17. Atlidakis, V.; Godefroid, P.; Polishchuk, M. Checking security properties of cloud service REST APIs. In Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 23–27 March 2020; pp. 387–397.
18. Godefroid, P.; Lehmann, D.; Polishchuk, M. Differential regression testing for REST APIs. In Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle, WA, USA, 17–21 July 2020; pp. 312–323.
19. Godefroid, P.; Huang, B.Y.; Polishchuk, M. Intelligent REST API data fuzzing. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, 8–13 November 2020; pp. 725–736.
20. Karlsson, S.; Čaušević, A.; Sundmark, D. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs. In Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Porto, Portugal, 23–27 March 2020; pp. 131–141. [\[CrossRef\]](#)
21. Hatfield-Dodds, Z.; Dygalo, D. Deriving Semantics-Aware Fuzzers from Web API Schemas. In Proceedings of the 2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), Pittsburgh, PA, USA, 22–24 May 2022; pp. 345–346. [\[CrossRef\]](#)
22. Tsai, C.H.; Tsai, S.C.; Huang, S.K. REST API Fuzzing by Coverage Level Guided Blackbox Testing. In Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS), Hainan, China, 6–10 December 2021; pp. 291–300. [\[CrossRef\]](#)
23. Martin-Lopez, A.; Segura, S.; Ruiz-Cortés, A. Test coverage criteria for RESTful web APIs. In Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation, A-TEST 2019, Tallinn, Estonia, 26–27 August 2019; Association for Computing Machinery: New York, NY, USA, 2019; pp. 15–21. [\[CrossRef\]](#)
24. Atlidakis, V.; Geambasu, R.; Godefroid, P.; Polishchuk, M.; Ray, B. Pythia: Grammar-based fuzzing of rest apis with coverage-guided feedback and learning-based mutations. *arXiv* **2020**, arXiv:2005.11498.
25. Arcuri, A. RESTful API Automated Test Case Generation. In Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS), Prague, Czech Republic, 25–29 July 2017; pp. 9–20. [\[CrossRef\]](#)
26. Arcuri, A. RESTful API Automated Test Case Generation with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* **2019**, *28*, 3. [\[CrossRef\]](#)
27. Arcuri, A.; Galeotti, J.P. Handling SQL Databases in Automated System Test Generation. *ACM Trans. Softw. Eng. Methodol.* **2020**, *29*, 22. [\[CrossRef\]](#)
28. Zhang, M.; Arcuri, A. Adaptive Hypermutation for Search-Based System Test Generation: A Study on REST APIs with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* **2021**, *31*, 2. [\[CrossRef\]](#)
29. Zhang, M.; Arcuri, A. Enhancing Resource-Based Test Case Generation for RESTful APIs with SQL Handling. In Proceedings of the Search-Based Software Engineering: 13th International Symposium, SSBSE 2021, Bari, Italy, 11–12 October 2021; Proceedings; Springer: Berlin/Heidelberg, Germany, 2021; pp. 103–117. [\[CrossRef\]](#)
30. Kim, M.; Xin, Q.; Sinha, S.; Orso, A. Automated test generation for rest apis: No time to rest yet. In Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual, 18–22 July 2022; pp. 289–301.
31. Martin-Lopez, A.; Segura, S.; Müller, C.; Ruiz-Cortés, A. Specification and Automated Analysis of Inter-Parameter Dependencies in Web APIs. *IEEE Trans. Serv. Comput.* **2022**, *15*, 2342–2355. [\[CrossRef\]](#)
32. Mirabella, A.G.; Martin-Lopez, A.; Segura, S.; Valencia-Cabrera, L.; Ruiz-Cortés, A. Deep Learning-Based Prediction of Test Input Validity for RESTful APIs. In Proceedings of the 2021 IEEE/ACM Third International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest), Madrid, Spain, 1 June 2021; pp. 9–16. [\[CrossRef\]](#)
33. Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; Neubig, G. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* **2023**, *55*, 195. [\[CrossRef\]](#)
34. Lemieux, C.; Inala, J.P.; Lahiri, S.K.; Sen, S. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-Trained Large Language Models. In Proceedings of the 45th International Conference on Software Engineering, ICSE '23, Melbourne, Australia, 17–19 May 2023; IEEE Press: Piscataway, NJ, USA, 2023; pp. 919–931. [\[CrossRef\]](#)
35. Deng, Y.; Xia, C.S.; Peng, H.; Yang, C.; Zhang, L. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023, Seattle, WA, USA, 17–21 July 2023; Association for Computing Machinery: New York, NY, USA, 2023; pp. 423–435. [\[CrossRef\]](#)
36. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pondé, H.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374.
37. Fried, D.; Aghajanyan, A.; Lin, J.; Wang, S.I.; Wallace, E.; Shi, F.; Zhong, R.; Tau Yih, W.; Zettlemoyer, L.; Lewis, M. InCoder: A Generative Model for Code Infilling and Synthesis. *arXiv* **2022**, arXiv:2204.05999.
38. Deng, Y.; Xia, C.S.; Yang, C.; Zhang, S.D.; Yang, S.; Zhang, L. Large Language Models are Edge-Case Generators: Crafting Unusual Programs for Fuzzing Deep Learning Libraries. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Lisbon, Portugal, 14–20 April 2024; Association for Computing Machinery: New York, NY, USA, 2024. [\[CrossRef\]](#)

39. Ackerman, J.; Cybenko, G. Large Language Models for Fuzzing Parsers (Registered Report). In Proceedings of the 2nd International Fuzzing Workshop, FUZZING 2023, Seattle, WA, USA, 17 July 2023; Association for Computing Machinery: New York, NY, USA, 2023; pp. 31–38. [\[CrossRef\]](#)
40. Meng, R.; Mirchev, M.; Böhme, M.; Roychoudhury, A. Large language model guided protocol fuzzing. In Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS), San Diego, CA, USA, 26 February–1 March 2024.
41. Cheng, M.; Zhu, K.; Chen, Y.; Yang, G.; Lu, Y.; Lu, C. MSFuzz: Augmenting Protocol Fuzzing with Message Syntax Comprehension via Large Language Models. *Electronics* **2024**, *13*, 2632. [\[CrossRef\]](#)
42. Wang, J.; Yu, L.; Luo, X. LLMIF: Augmented Large Language Model for Fuzzing IoT Devices. In Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP). IEEE Computer Society, San Francisco, CA, USA, 23 May 2024; p. 196.
43. Xia, C.S.; Paltenghi, M.; Le Tian, J.; Pradel, M.; Zhang, L. Fuzz4All: Universal Fuzzing with Large Language Models. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE'24, Lisbon, Portugal, 14–20 April 2024; Association for Computing Machinery: New York, NY, USA, 2024. [\[CrossRef\]](#)
44. Decrop, A.; Perrouin, G.; Papadakis, M.; Devroey, X.; Schobbens, P.Y. You Can REST Now: Automated Specification Inference and Black-Box Testing of RESTful APIs with Large Language Models. *arXiv* **2024**, arXiv:2402.05102.
45. Kim, M.; Stennett, T.; Shah, D.; Sinha, S.; Orso, A. Leveraging Large Language Models to Improve REST API Testing. In Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, ICSE-NIER'24, Lisbon, Portugal, 14–20 April 2024; Association for Computing Machinery: New York, NY, USA, 2024; pp. 37–41. [\[CrossRef\]](#)
46. Richards, R.; Richards, R. Representational state transfer (rest). In *Pro PHP XML and Web Services*; Apress: New York, NY, USA, 2006; pp. 633–672.
47. Fielding, R.T. *Architectural Styles and the Design of Network-Based Software Architectures*; University of California: Irvine, CA, USA, 2000.
48. What is REST? Available online: <https://restfulapi.net/> (accessed on 1 July 2024).
49. Berners-Lee, T.; Fielding, R.; Frystyk, H. *Hypertext transfer protocol-HTTP/1.0*; RFC Editor: Marina del Rey, CA, USA, 1996; Volume 1945, pp. 1–60.
50. Thoppilan, R.; De Freitas, D.; Hall, J.; Shazeer, N.; Kulshreshtha, A.; Cheng, H.T.; Jin, A.; Bos, T.; Baker, L.; Du, Y.; et al. Lamda: Language models for dialog applications. *arXiv* **2022**, arXiv:2201.08239.
51. Ye, Q.; Axmed, M.; Pryzant, R.; Khani, F. Prompt engineering a prompt engineer. *arXiv* **2023**, arXiv:2311.05661.
52. HTTP Methods. Available online: <https://restfulapi.net/http-methods/> (accessed on 1 July 2024).
53. Shelby, Z. RFC 6690: *Constrained RESTful Environments (CoRE) Link Format*; RFC Editor: Marina del Rey, CA, USA, 2012; pp. 1–20.
54. Nadkarni, P.M.; Ohno-Machado, L.; Chapman, W.W. Natural language processing: An introduction. *J. Am. Med Inform. Assoc.* **2011**, *18*, 544–551. [\[CrossRef\]](#) [\[PubMed\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.